

PROJECT DISPATCH — BUILD PLAN

News & Article Platform Architecture and Security Blueprint

A full-featured news publishing platform: admins and moderators author and manage articles with photo, video and link embeds; readers register, comment, react and share to social media. This document specifies the stack, data model, feature set, and — as a first-class concern — how the system is hardened against attackers.

Next.js 15 (App Router)

PostgreSQL + Prisma

Auth.js

TypeScript, strict

Prepared 2026-09-16

CONTENTS

01 Overview & goals

02 System architecture

03 Roles & permissions

04 Data model

05 Core features

06 Security blueprint

07 Infrastructure & deployment

08 Project structure

09 Build roadmap

10 Testing & QA

11 Appendix: key packages

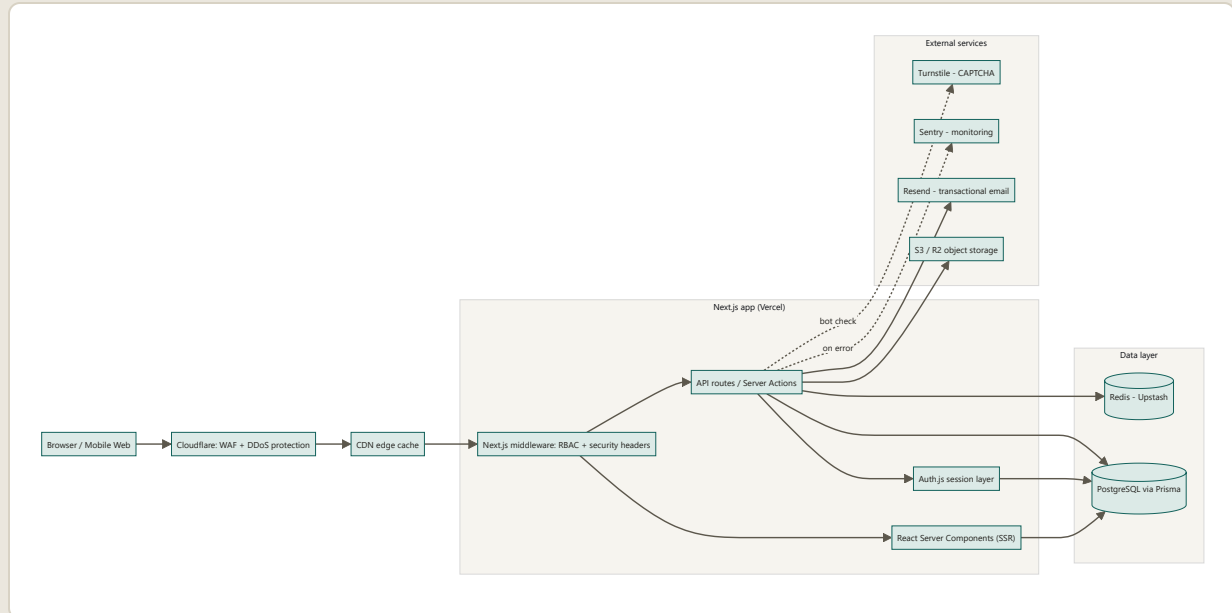
01 Overview & goals

The platform has two audiences with very different needs: a small internal **newsroom** (admins, moderators) publishing time-sensitive content, and a large public **readership** that consumes, comments on, and redistributes that content. The plan below is scoped end-to-end — architecture, data, features, and hardening — for a full-featured build, sequenced into shippable phases.

Design priorities, in order: (1) content integrity — only authorized staff can publish or alter what's on the site; (2) reader trust & safety — comments and accounts can't be trivially abused; (3) availability — the site stays up and fast under real news-traffic spikes; (4) reach — content is easy to find, read, and share.

02 System architecture

Next.js (App Router) is recommended as the single full-stack codebase: React Server Components render article pages on the server (fast, good SEO — essential for a news site's organic traffic), while API routes / Server Actions handle mutations. **PostgreSQL** via **Prisma ORM** is the system of record; **Redis (Upstash)** backs rate limiting, session/token caches, and view counters; media lives in **S3-compatible object storage**, never on the app server's disk.



Why this shape: putting **all authorization checks in one middleware layer plus server-side handlers** (never in client components alone) means there is one place to audit for access-control bugs. Putting Cloudflare in front absorbs volumetric attacks before they reach the app at all — important for a news site, which is a predictable target during controversial stories.

03 Roles & permissions

Four roles, enforced **server-side on every request** — a role claim in a cookie or JWT is never trusted by itself; each handler re-checks the user's current role against the database.

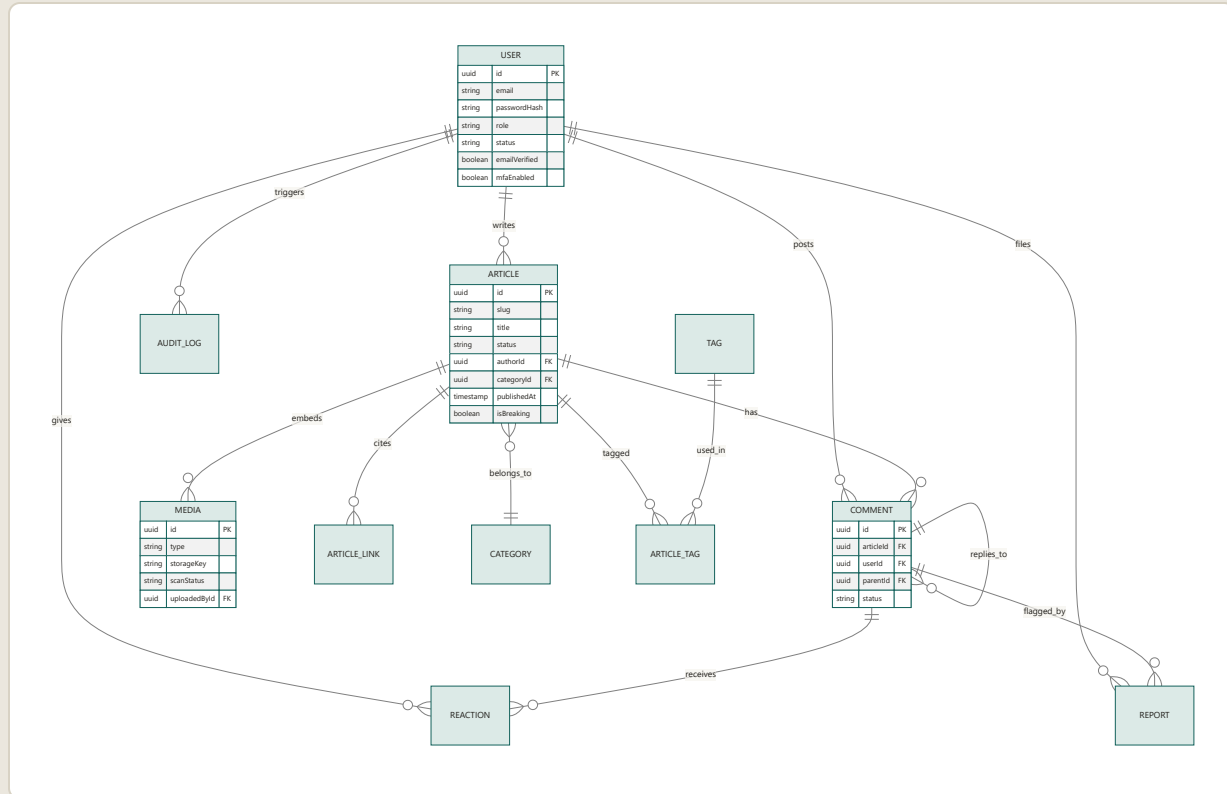
ACTION	GUEST	READER	MODERATOR	ADMIN
Read published articles, share to social	✓	✓	✓	✓
Register, comment, react, bookmark, follow	—	✓	✓	✓
Write & edit own draft articles, upload photo/video/links	—	—	✓	✓
Publish / schedule / unpublish articles	—	—	✓	✓
Edit or unpublish <i>another</i> author's article	—	—	—	✓
Approve / hide / delete comments, mute or ban readers	—	—	✓	✓
Manage categories, tags, homepage layout	—	—	—	✓
Manage staff accounts & roles	—	—	—	✓
View analytics dashboard	—	—	✓	✓
View audit log, change site & security settings	—	—	—	✓

RECOMMENDED REFINEMENT

As the newsroom grows past a handful of people, split "Moderator" into **Editor** (writes/publishes content, no user-management power) and **Moderator** (manages comments/reports, no publishing power). The schema below already stores role as an extensible enum so this is a config change, not a rewrite.

04 Data model

Core entities and relationships. Every mutable content table carries a **status** and timestamps so nothing is destructively deleted from the reader's perspective — comments and articles are soft-removed (status flag) and remain in the audit trail.



Notable design choices

- **Article body** is stored as structured JSON (from a Tiptap/Lexical rich-text editor), not raw HTML from the client — HTML is generated server-side from the JSON through a strict sanitizer allowlist at render time. This closes off stored-XSS via the editor.
- **Media** rows track **scanStatus** (pending / clean / flagged) — an uploaded file is not publicly served until it passes processing and a malware scan.
- **Comment** is self-referencing via **parentId** for threaded replies, and carries a **status** (pending / approved / hidden / deleted) so moderation never hard-deletes user content pre-review.
- **AuditLog** is append-only: every admin/moderator action (publish, delete, ban, role change) writes a row with actor, action, target, IP, and timestamp. It has no update or delete API — only inserts.

05 Core features

Newsroom (Admin / Moderator)

- Rich-text article editor with inline photo/video embeds, pull quotes, and external link cards (auto-fetches title/preview via a server-side proxy, never client-side, to avoid leaking reader IPs to third parties)
- Draft → In Review → Scheduled → Published → Archived workflow with per-transition permission checks
- Direct-to-storage media upload (photo/video) with progress, client-side type/size pre-check, and server-side re-validation
- Breaking-news flag that triggers push/email alerts to subscribed readers
- Category, tag, and homepage/section layout management
- Comment moderation queue: approve, hide, delete, ban author, with bulk actions
- Analytics dashboard: views, read-time, referrers, top shares, comment volume
- Full audit log viewer (admin only)

Reader experience

- Register / log in (email+password, optional Google OAuth), email verification, password reset
- Comment with nested replies, like/react, report abusive comments
- Follow authors, categories, or topics; save/bookmark articles for later
- One-click share to X, Facebook, WhatsApp, LinkedIn, email, and copy-link — via Open Graph / Twitter Card meta tags so shared links render a rich preview
- Full-text search with filters (category, date, author)
- Optional push/email notifications for breaking news and followed topics
- Account settings: profile, notification preferences, connected devices/sessions (with "log out everywhere"), data export/delete

Platform-wide

- SEO: server-rendered pages, sitemap.xml, RSS feed, JSON-LD `NewsArticle` structured data
- Accessibility: semantic HTML, keyboard navigation, alt text required on images at upload

- i18n-ready content model (locale field on Article) even if only one locale ships at launch

06 Security blueprint

A news site is a realistic target: defacement is reputationally catastrophic, comment sections attract spam/abuse at scale, and breaking stories bring traffic spikes attackers can hide inside. Security is treated as a feature with its own acceptance criteria, not a pass at the end.

Authentication & sessions

- Passwords hashed with **Argon2id** (not bcrypt/MD5/SHA); minimum length 12, checked against the HavelBeenPwned breached-password API (k-anonymity, no plaintext leaves the server)
- **Auth.js** with database-backed sessions (not stateless-only JWTs) so any session can be instantly revoked — required for "log out everywhere" and for killing a compromised account
- Session cookies: `httpOnly`, `secure`, `sameSite=lax`, short expiry with sliding renewal
- Exponential backoff + account lockout after repeated failed logins, tracked per account *and* per IP so an attacker can't just rotate accounts to dodge the lockout
- **Mandatory TOTP-based MFA for Admin accounts**; optional (encouraged) for Moderators and readers
- Email verification required before an account can publish, comment, or upload — cuts off the cheapest bot-signup path

Authorization (RBAC)

- Every API route / Server Action re-checks role and resource ownership **server-side** — a hidden admin button on the client is not a security control, only a UX one
- Centralized middleware gates all `/dashboard/*` and admin API routes before they execute
- Principle of least privilege: Moderators can unpublish but not permanently delete; only Admins can change roles or delete accounts

Input handling & injection defense

- Every input validated server-side with **Zod schemas** — client-side validation is UX sugar only, never trusted
- All database access through Prisma's parameterized queries; no raw string-concatenated SQL, anywhere

- Rich article content is sanitized server-side with a strict tag/attribute allowlist (DOMPurify on the server) both on save *and* on render — defense in depth in case a bad row ever gets in
- React's default escaping is relied on everywhere; `dangerouslySetInnerHTML` is used in exactly one place (the sanitized article renderer) and nowhere else in the codebase

FILE UPLOAD ATTACK SURFACE

Uploads are the highest-risk feature on this kind of site, so they get their own hardened path:

- Files go **client** → **pre-signed URL** → **object storage directly**, never through the app server's own disk or memory
- Type is verified by magic bytes, not filename/extension; size capped per media type
- Images are re-encoded (via `sharp`) and videos transcoded — this strips embedded scripts/EXIF and neutralizes polyglot-file attacks
- Every upload is malware-scanned before its `scanStatus` flips to "clean" and it becomes publicly reachable
- Media is served from a separate, cookie-less subdomain/CDN so a compromised media file can't ride the main site's session cookies

Network & browser-side defenses

- Strict **Content-Security-Policy** (no `unsafe-inline` scripts; nonces for the few inline scripts that are unavoidable)
- HSTS with preload, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, locked-down `Permissions-Policy`
- HTTPS enforced everywhere; HTTP requests redirected at the edge
- Cloudflare (or equivalent) in front for WAF rules and DDoS absorption before traffic reaches the app

Rate limiting & abuse prevention

- Redis-backed rate limits on login, registration, comment posting, password reset, and search — tuned separately per endpoint
- Cloudflare Turnstile (CAPTCHA) on registration and after repeated login failures

- New accounts' first comments enter the moderation queue by default; trust is earned over time (fewer restrictions after N approved comments)
- Link-density / keyword heuristics flag likely spam for moderator review rather than auto-posting it

Secrets, dependencies & supply chain

- All secrets live in the hosting platform's environment/secret manager, per environment; nothing sensitive is ever committed — only a `.env.example` ships in the repo
- Dependabot / `npm audit` wired into CI; patches for auth- and sanitization-related packages are treated as urgent
- Any third-party script (analytics, share widgets) loads with Subresource Integrity where possible, and is reviewed before adoption

Logging, monitoring & audit trail

- Append-only **audit log** for every privileged action (publish, delete, role change, ban) — actor, action, target, IP, timestamp, never editable
- Centralized error tracking (Sentry) with PII scrubbing before events leave the server
- Alerting on anomalies: spikes in failed logins, 5xx rates, or comment-posting velocity

Data protection & incident response

- TLS in transit; encryption at rest via the managed Postgres provider
- PII minimization — only what's operationally needed is stored, with a data export/delete flow for user requests
- Automated daily backups with point-in-time recovery, and a periodically *tested* restore drill (an untested backup is not a backup)
- Written incident runbook: detect → contain (revoke sessions, rotate secrets) → eradicate → recover → post-mortem

OWASP Top 10 (2021) — mapping

RISK	MITIGATION IN THIS DESIGN
A01 Broken access control	Server-side RBAC on every route; centralized middleware; resource-ownership checks
A02 Cryptographic failures	Argon2id hashing, TLS everywhere, encrypted-at-rest DB, no secrets in code/repo
A03 Injection	Prisma parameterized queries; Zod validation on every input; sanitized rich-text rendering
A04 Insecure design	Threat-modeled upload path, moderation-by-default for new accounts, least-privilege roles
A05 Security misconfiguration	CSP/HSTS/security headers via middleware; environment-scoped secrets; hardened defaults, no debug endpoints in prod
A06 Vulnerable components	Dependabot + audit in CI; minimal, vetted dependency set for auth/sanitization
A07 Auth & session failures	Revocable DB sessions, logout/backoff, MFA for staff, breached-password checks
A08 Data integrity failures	Signed, scanned, re-encoded uploads; append-only audit log; CI-gated deploys
A09 Logging & monitoring failures	Structured audit log, Sentry error tracking, anomaly alerting
A10 Server-side request forgery	Link-preview fetcher runs server-side with an allowlisted/validated URL scheme and no access to internal network ranges

07 Infrastructure & deployment

LAYER	CHOICE	WHY
Hosting	Vercel	First-class Next.js support, automatic HTTPS/edge network, per-PR preview deploys
Database	Managed Postgres (Neon / RDS)	Automated backups, point-in-time recovery, connection pooling
Media storage	Cloudflare R2 or S3 + CDN	Cookie-less origin, cheap egress, direct-upload support
Cache / rate limiting	Upstash Redis	Serverless-friendly, low-latency counters for limits and view counts
Email	Resend / SES	Transactional email for verification, reset, notifications
Search	Postgres full-text search → Meilisearch if scaling	Start simple; swap in a dedicated engine only once volume justifies it
Edge security	Cloudflare (proxy mode)	WAF, DDoS mitigation, Turnstile CAPTCHA
CI/CD	GitHub Actions	Lint, typecheck, test, dependency audit gate every merge; preview per PR

Environments: local → per-PR preview → staging (mirrors prod, used to test security patches safely) → production.

08 Project structure

```
app/  
  (public)/           home, category pages, /article/[slug], /author/[handle], /search/  
  (auth)/             login, register, verify-email, reset-password  
  (dashboard)/        admin + moderator area, gated by middleware  
    articles/ media/ comments/ users/ analytics/ settings/  
  api/                route handlers not covered by server actions  
  
components/  
  article/ editor/ comment/ layout/ ui/  
  
lib/  
  auth.ts             Auth.js config  
  db.ts               Prisma client singleton  
  validation/         Zod schemas, one per resource  
  sanitize.ts         server-side HTML sanitizer allowlist  
  rateLimit.ts        Upstash-backed limiters  
  storage.ts          pre-signed upload URL issuance  
  email.ts            transactional email senders  
  
prisma/  
  schema.prisma  
  migrations/  
  
middleware.ts         RBAC gate + security headers, runs before every request  
tests/  
  unit/ integration/ e2e/ (Playwright)
```

09 Build roadmap

Full feature set, sequenced so every phase ships something runnable and security is built in from phase 1, not bolted on at the end.

Phase 0 Foundation

Repo, Next.js + TypeScript strict mode, Prisma schema, CI pipeline (lint/typecheck/test/audit), staging environment.

Phase 1 Identity & access

Registration, login, email verification, password reset, Auth.js sessions, RBAC middleware, audit log table. Nothing else ships until this is solid.

Phase 2 Article authoring

Rich-text editor, draft/publish workflow, direct-to-storage photo/video upload with scanning + re-encoding, link embeds, categories/tags.

Phase 3 Public reading experience

Home/category/article pages (SSR), SEO metadata, Open Graph share cards, sitemap/RSS.

Phase 4 Engagement

Comments with threading, moderation queue, reactions, bookmarks, follows, social share buttons, rate limiting + CAPTCHA on write actions.

Phase 5 Newsroom tooling

Admin dashboard: analytics, comment moderation UI, user/role management, audit log viewer, breaking-news alerts.

Phase 6 Hardening & launch prep

Full OWASP Top 10 pass, CSP/security headers finalized, MFA for staff, backup/restore drill, external security review, load test.

Phase 7 Post-launch

Search upgrade, push notifications, i18n, personalization — layered on once the core is stable in production.

10 Testing & QA

- **Unit tests** (Vitest): validation schemas, sanitizer allowlist, permission-check helpers
- **Integration tests**: every API route tested for the happy path *and* for unauthorized access — a Reader token hitting an Admin route must fail, every time, as a regression gate
- **End-to-end** (Playwright): register → verify → login, author writes → publishes → reader comments → shares, moderator hides a comment
- **Automated security scan** in CI (OWASP ZAP baseline) plus a manual penetration test pass before launch
- **Accessibility checks** on core templates (article page, comment form, dashboard)

11 Appendix: key packages

CONCERN	PACKAGE
Framework	next, react, typescript
Database	prisma, @prisma/client
Auth	next-auth (Auth.js), argon2
Validation	zod
Sanitization	dompurify, jsdom
Rich text editor	@tiptap/react or lexical
Image processing	sharp
Rate limiting	@upstash/ratelimit, @upstash/redis
Email	resend
Monitoring	@sentry/nextjs
Testing	vitest, @playwright/test

Project Dispatch — architecture & security plan. Prepared as a starting blueprint; role split, search engine, and notification stack are the easiest levers to revisit once real traffic patterns are known.